SmartLogix

Conception et developpement d'une plateforme intelligente de gestion logistique

Rapport de Projet de Fin d'Etudes (PFE)

Auteur : [A completer]

Formation : [A completer]

Etablissement : [A completer]

Encadrant : [A completer]

Annee universitaire : 2025-2026

Nom technique depot : SmartTask

Technologies : Java 17, Spring Boot 4.1, Angular 16, PostgreSQL, Python FastAPI, scikit-learn, XGBoost, JWT, Flyway

Date : 01/09/2026

Table des matieres

1. Remerciements et resume executif
2. Cartographie du projet
3. Chapitre 1 - Contexte general du projet
4. Chapitre 2 - Analyse des besoins
5. Chapitre 3 - Methodologie de developpement
6. Chapitre 4 - Architecture du systeme
7. Chapitre 5 - Diagrammes UML (cas d'utilisation)
8. Chapitre 6 - Diagramme de classes
9. Chapitre 7 - Base de donnees
10. Chapitre 8 - Backend Spring Boot
11. Chapitre 9 - Frontend Angular
12. Chapitre 10 - Intelligence artificielle et Machine Learning
13. Chapitre 11 - Diagrammes de sequence
14. Chapitre 12 - Outils et technologies
15. Chapitre 13 - Tests
16. Chapitre 14 - Securite
17. Chapitre 15 - Docker et DevOps
18. Chapitre 16 - Deploiement
19. Chapitre 17 - Scenario global de fonctionnement
20. Chapitre 18 - Difficultes et solutions
21. Chapitre 19 - Choix technologiques
22. Chapitre 20 - Limites et perspectives
23. Chapitre 21 - Conclusion
24. Annexes

Liste des figures

Figure 4.1 - Architecture generale du systeme
Figure 4.2 - Architecture logique en couches
Figure 4.3 - Architecture physique (ports et services)
Figure 5.1 - Diagramme de cas d'utilisation global
Figure 6.1 - Diagramme de classes metier
Figure 7.1 - Modele entite-relation (ERD)
Figure 10.1 - Architecture du service IA
Figure 10.2 - Flux prediction Machine Learning
Figure 11.1 - Sequence authentification JWT
Figure 11.2 - Sequence creation commande + automation
Figure 11.3 - Sequence prediction IA
Figure 11.4 - Sequence optimisation itineraire chauffeur
Figure 11.5 - Sequence chatbot Copilot
Figure 16.1 - Schema de deploiement possible

Liste des tableaux

Tableau 1.1 - Critique de l'existant
Tableau 1.2 - Problemes et solutions SmartLogix
Tableau 2.1 - Acteurs du systeme
Tableau 2.2 - Besoins non fonctionnels
Tableau 3.1 - Phases de developpement
Tableau 7.1 - Tables PostgreSQL (Flyway V1-V10)
Tableau 8.1 - Technologies backend
Tableau 8.2 - Controllers REST (13 controllers, 97 endpoints)
Tableau 8.3 - Catalogue complet des API REST
Tableau 9.1 - Routes Angular par role
Tableau 10.1 - Variables du dataset ML
Tableau 10.2 - Comparaison modeles ML
Tableau 10.3 - Chaine automation 7 etapes
Tableau 10.4 - Endpoints Python FastAPI
Tableau 12.1 - Stack technologique complete
Tableau 13.1 - Strategie de tests
Tableau 18.1 - Difficultes rencontrees
Annexe - Comptes de test et types produit

RESUME EXECUTIF

SmartLogix est une plateforme logistique intelligente developpee en Java Spring Boot, Angular et Python. Elle dessert 5 roles (Admin, Manager, Operator, Driver, Client) et couvre le cycle complet commande-livraison-avis.

Architecture : Angular (4200) -> Spring Boot (9090) -> PostgreSQL + Python FastAPI (8000). 91 endpoints REST, 11 entites JPA, 10 migrations Flyway, 49 composants Angular.

Intelligence artificielle : prediction retard (F1=0.982), ETA (MAE=6.3 min), chaine automation 7 etapes, chatbot Copilot, optimisation itineraire, alertes chauffeur.

Securite JWT, RBAC, BCrypt. Tests : JUnit (7 fichiers), pytest (14 tests). Docker partiel (service IA). Perspectives : CI/CD, donnees reelles ML, mobile chauffeur.

CARTOGRAPHIE DU PROJET (verification code source)

Composant	Nombre	Detail
Controllers REST Java	13	91 endpoints documentes
Services Java	21+	automation, chat, ai, driver, metier
Entities JPA	11	User, Task, Order, ProductType...
Repositories	11	Spring Data JPA
Migrations Flyway	10	V1 a V10
Composants Angular	49	6 roles + shared + chat widget
Services Angular	11	HTTP vers API Spring Boot
Endpoints Python IA	11	FastAPI /api/ai/*
Tests Java	7 fichiers	20 methodes @Test
Tests Python	2 modules	14 tests pytest
Tests Angular	1 spec	app.component.spec.ts

Note : le titre du rapport est SmartLogix. Le depot source utilise le nom technique SmartTask. Les deux designent le meme projet.

CHAPITRE 1 - CONTEXTE GENERAL DU PROJET

1.1 Introduction

SmartLogix est une plateforme web de gestion logistique destinee aux entreprises de livraison et de transport. Elle digitalise le cycle complet : prise de commande, preparation, assignation chauffeur, execution de la livraison, confirmation client, gestion des incidents et tableaux de bord par role.

Le projet s'inscrit dans une demarche de modernisation des systemes d'information logistiques, en integrant une architecture distribuee et des capacites d'intelligence artificielle pour l'aide a la decision et l'automatisation.

1.2 Contexte du projet

Domaine : logistique du dernier kilometre (last-mile delivery). Contexte academique : Projet de Fin d'Etudes (PFE) en genie informatique. Le secteur connait une croissance des livraisons express, imposant reactivite, tracabilite et coordination multi-acteurs.

- Besoin de centraliser commandes, taches et livraisons
- Necessite d'automatiser les decisions pour les urgences HIGH/CRITICAL
- Integration IA pour prediction, recommandation et assistance

1.3 Problematique

Comment concevoir une plateforme multi-acteurs integrant l'intelligence artificielle pour automatiser les livraisons urgentes, predire les retards, optimiser les itineraires chauffeur et assister les utilisateurs via un copilote conversationnel, tout en garantissant securite, tracabilite et experience utilisateur par role ?

1.5 Critique de l'existant

Probleme	Consequence	Limitation
Traitement manuel commandes	Lenteur, erreurs	Pas de workflow
Assignation chauffeur intuitive	Sous-optimal	Pas de scoring
Retards non anticipes	Insatisfaction	Reactif seulement
Suivi fragmente	Manque visibilite	Communication difficile
Saisie libre produits	Donnees incoherentes	Pas de standard
Pas d'assistant centralise	Charge support	FAQ statiques

1.6 Solution proposee

Probleme	Solution SmartLogix
Commandes manuelles	Workflow Operator + catalogue types produit
Assignation	IA recommandation + automation express
Retards	Prediction ML + alertes proactives chauffeur
Suivi	SSE temps reel + notifications
Typage produits	6 types + option Autre
Assistance	Chatbot SmartTask Copilot par role
Urgences	Chaine automation 7 etapes auto HIGH/CRITICAL

1.7 Objectifs du projet

Objectifs fonctionnels

- Gerer 5 roles : Admin, Manager, Operator, Driver, Client

- Cycle commande -> tache -> livraison -> avis client
- Incidents, notifications, analytics manager, export CSV
- Types de produits standardises avec saisie libre optionnelle

Objectifs

- Architecture 3 tiers : Angular + Spring Boot + Python FastAPI
- API REST securisee JWT, migrations Flyway, validation Bean Validation
- Microservice IA decouple via WebClient HTTP

Objectifs

- Prediction retard (classification) et ETA (regression)
- Recommandation chauffeur, priorisation taches
- Automation express, optimisation route, alertes proactives
- Chatbot contextuel avec historique persistant

Objectifs

- Authentification JWT stateless, mots de passe BCrypt
- Controle d'acces base sur les roles (RBAC)

Objectifs

- Dockerfile pour service IA (partiel)

Fonctionnalite prevue / future amelioration : CI/CD complet, docker-compose stack, Kubernetes.

1.8 Valeur ajoutee

SmartLogix combine gestion metier complete et IA operationnelle : predictions, automation autonome, copilote conversationnel et optimisation itineraire. L'architecture microservices permet d'evoluer le module IA independamment du backend.

1.9 Organisation du rapport

Le rapport presente successivement l'analyse des besoins, l'architecture, les diagrammes UML, la base de donnees, le backend, le frontend, l'IA en profondeur, les tests, la securite, le deploiement et les perspectives.

CHAPITRE 2 - ANALYSE DES BESOINS

2.1 Identification des acteurs

Acteur	Description	Responsabilites
ADMIN	Administrateur systeme	CRUD utilisateurs, roles, activation
MANAGER	Superviseur logistique	Taches, chauffeurs, IA, analytics, automation
OPERATOR	Operateur entrepot	Commandes, validation, preparation, incidents
DRIVER	Chauffeur livreur	Missions, livraison, GPS, itineraire IA, alertes
CLIENT	Client final	Suivi livraison SSE, confirmation, avis

2.2 Besoins fonctionnels (implementes dans le code)

- Authentification JWT (register, login, logout, me)
- Gestion utilisateurs complete (Admin)
- Gestion commandes avec types produit et option Autre (Operator)
- Workflow commande : PENDING -> VERIFIED -> READY -> IN_DELIVERY -> DELIVERED -> RECEIVED
- Gestion taches et assignation chauffeur (Manager)
- Cycle mission chauffeur : accept, start, delivery, complete, proof
- Suivi livraison SSE temps reel (Client)
- Incidents multi-acteurs avec traitement manager
- Avis clients (rating 1-5, commentaire, unique par tache)
- Notifications par utilisateur avec marquage lu
- Analytics manager + export CSV
- Predictions IA : retard, ETA, priorite, chauffeur, anomalie
- Chaine automation Livraison Express (7 etapes)
- Chatbot SmartTask Copilot avec historique
- Optimisation itineraire chauffeur via IA
- Alertes IA proactives chauffeur (scheduler 5 min)

2.3 Besoins non fonctionnels

NFR	Implementation observee
Securite	JWT, BCrypt, RBAC @PreAuthorize, CORS
Performance	Index BDD V6, pagination, open-in-view=false
Maintenabilite	Couches Controller/Service/Repository/DTO
Evoluabilite	Microservice IA separe, events async
Disponibilite IA	Fallback rule-based si Python indisponible
Testabilite	JUnit, pytest, profil test H2
Ergonomie	Interfaces par role, chat widget, feedback visuel

2.4 Contraintes

- Techniques : 3 services a demarrer (Spring, Angular, Python)
- Logicielles : PostgreSQL requis, modeles .pkl a generer localement
- Securite : secret JWT a changer en production
- Materielles : Information non identifiee dans les fichiers fournis.
- Temporelles : Information non identifiee dans les fichiers fournis.

CHAPITRE 3 - METHODOLOGIE DE DEVELOPPEMENT

3.1 Methodologie adoptee

Developpement iteratif et incremental, organise en phases fonctionnelles successives. Chaque phase livre un increment testable. Outils : Git, Gradle, npm, Flyway, Swagger.

3.2 Organisation du travail

- Backlog fonctionnel par module (auth, metier, IA, automation)
- Integration continue manuelle : build Gradle + ng build + pytest
- Tests unitaires et tests manuels via Swagger/Postman

3.3 Planification

Phase	Objectif	Livrables
P1	Auth + users + 5 roles	JWT, guards Angular
P2	Metier logistique	Orders, Tasks, Drivers, Notifications
P3	Frontend multi-role	49 composants, routing lazy
P4	Microservice IA	FastAPI, train ML, .pkl
P5	Integration IA	WebClient, page Manager IA
P6	Chatbot Copilot	V8 chat_messages, widget
P7	Automation express	V9 automation_logs, 7 etapes
P8	Types produit	V10 product_types, formulaire Operator
P9	IA Chauffeur	Route optimize + alertes scheduler

CHAPITRE 4 - ARCHITECTURE DU SYSTEME

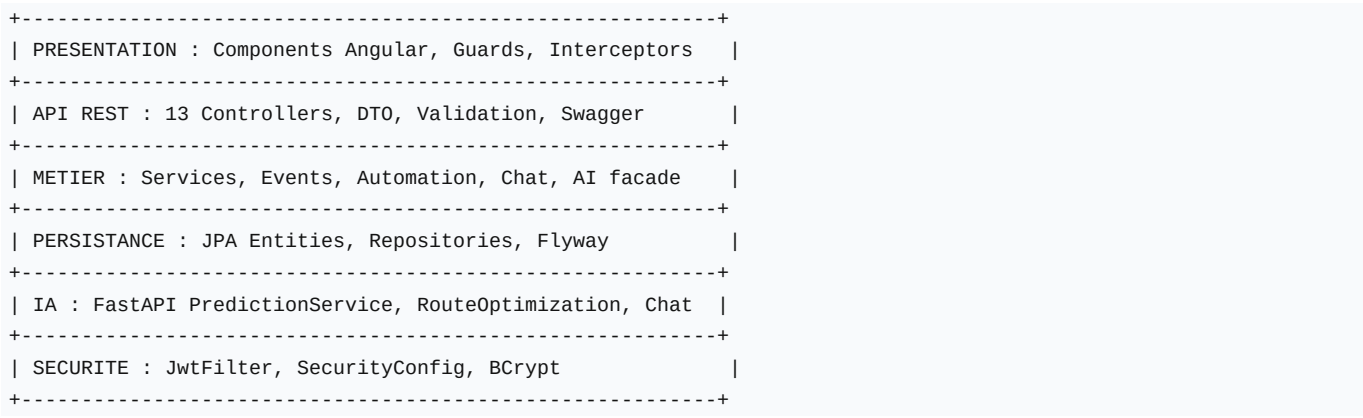
Figure 4.1 - Architecture generale



4.2 Architecture logique

Couche presentation (Angular) | Couche API REST (Controllers) | Couche metier (Services) | Couche persistance (Repositories/JPA) | Couche IA (Python microservice) | Couche securite transversale (JWT, RBAC).

Figure 4.2 - Architecture logique en couches



4.3 Architecture physique

Composant	Port	Technologie	Deploiement
Frontend	4200	Angular dev server	Static build possible
Backend	9090	Spring Boot embedded Tomcat	JAR executable
IA	8000	Uvicorn + FastAPI	Dockerfile present
BDD	5432	PostgreSQL	Service externe

4.4 Architecture applicative backend

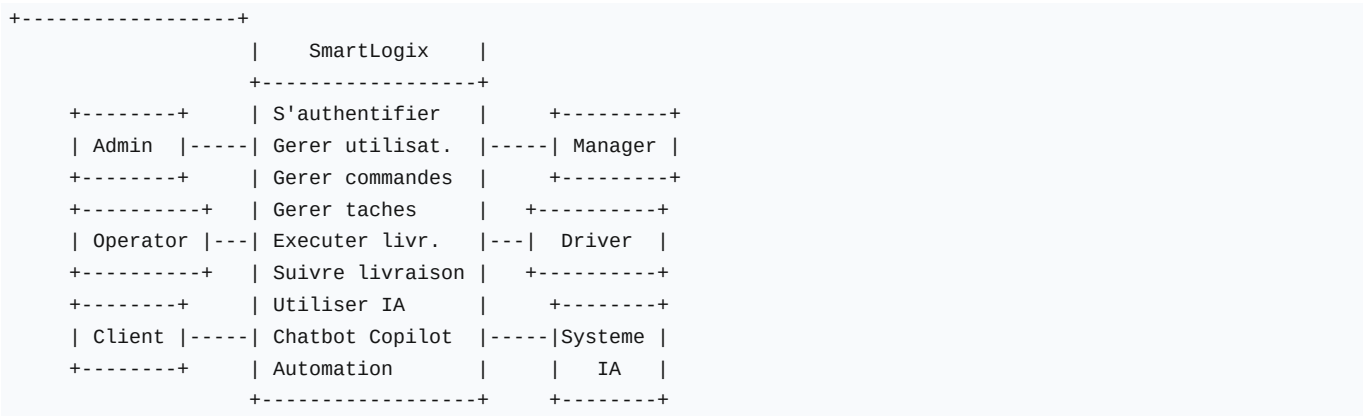
Pattern en couches avec inversion de controle Spring : Controller recoit requetes HTTP, delegue au Service pour la logique metier, qui utilise Repository pour la persistance. DTO separent le modele API du modele JPA. GlobalExceptionHandler centralise les erreurs.

4.5 Architecture microservices

Le projet adopte une architecture distribuee legere : le backend Spring Boot est le orchestrateur central ; le microservice Python IA est stateless et communique uniquement via HTTP REST JSON. Le frontend ne contacte jamais Python directement.

CHAPITRE 5 - DIAGRAMMES UML - CAS D'UTILISATION

Figure 5.1 - Cas d'utilisation global



5.2 Cas d'utilisation par acteur

Acteur	Cas d'utilisation	Interdit
ADMIN	CRUD users, roles, status	Gestion commandes/taches
MANAGER	Dashboard, taches, IA, automation, analytics	Creer commandes
OPERATOR	Commandes, verify, prepare, incidents	Assigner chauffeurs
DRIVER	Missions, livraison, route IA, alertes	Gerer autres chauffeurs
CLIENT	Suivi SSE, confirm, avis	Acces manager/operator

5.3 Descriptions textuelles des cas d'utilisation

CU : S'authentifier

Acteur : Tous les acteurs | Precondition : Compte actif existant

Declencheur : Soumission formulaire login

Scenario nominal : Saisir email/mot de passe -> POST /api/auth/login -> recevoir JWT -> redirection dashboard role

Scenarios alternatifs : Mot de passe incorrect -> message erreur 401

Postcondition : Session JWT active 24h

CU : Creer une commande

Acteur : OPERATOR | Precondition : Authentifie OPERATOR, types produit disponibles

Declencheur : Formulaire nouvelle commande

Scenario nominal : Selectionner client, type produit ou Autre, priorite -> POST /api/operator/orders -> commande PENDING

Scenarios alternatifs : Priorite HIGH -> declenchement automation via OrderCreatedEvent

Postcondition : Commande enregistree, notification client

CU : Executer chaine automation express

Acteur : Systeme (automatique) ou MANAGER/OPERATOR (manuel) | Precondition : Commande HIGH/CRITICAL, automation enabled

Declencheur : OrderCreatedEvent ou POST /api/automation/express/{id}

Scenario nominal : 7 etapes: READ_ORDER, AUTO_VERIFY, CREATE_TASK, AI_PRIORITIZE, AI_ASSIGN_DRIVER, ESTIMATE_AND_NOTIFY, LOG_COMPLETE

Scenarios alternatifs : Python IA indisponible -> fallback regles, etape SKIPPED ou FAILED loguee

Postcondition : Tache creee, journal automation_logs complet

CU : Predire risque de retard

Acteur : MANAGER | Precondition : Tache existante, service IA actif

Declencheur : Clic analyser sur page Manager IA

Scenario nominal : POST /api/manager/ai/predict-delay -> WebClient Python -> delay_model.pkl -> probabilite + niveau risque

Scenarios alternatifs : Timeout Python -> RuleBasedAIRecommendationService

Postcondition : Resultat affiche dans interface manager

CU : Optimiser itineraire chauffeur

Acteur : DRIVER | Precondition : Plusieurs missions actives

Declencheur : Acces page Route chauffeur

Scenario nominal : GET /api/driver/route/optimized -> Python POST /api/ai/optimize/route -> ordre optimal TSP

Scenarios alternatifs : Une seule mission -> ordre trivial

Postcondition : Liste missions reordonnee affichee

CU : Suivre livraison temps reel

Acteur : CLIENT | Precondition : Commande en livraison

Declencheur : Acces page suivi livraison

Scenario nominal : GET /api/client/deliveries/{id}/stream (SSE) -> mises a jour statut/GPS

Scenarios alternatifs : Connexion SSE perdue -> reconnexion automatique

Postcondition : Client informe en temps reel

CU : Confirmer reception et laisser avis

Acteur : CLIENT | Precondition : Livraison DELIVERED

Declencheur : Confirmation reception

Scenario nominal : POST confirm -> POST review rating 1-5 + commentaire

Scenarios alternatifs : Avis deja existant -> erreur contrainte unique

Postcondition : Commande RECEIVED, avis enregistre

CU : Utiliser chatbot Copilot

Acteur : Tous (authentifies) | Precondition : Session chat initialisee

Declencheur : Message dans ChatWidget

Scenario nominal : POST /api/chat/message -> ChatService -> Python /api/ai/chat ou fallback Java

Scenarios alternatifs : Intent non reconnu -> reponse generique knowledge base

Postcondition : Message sauve en chat_messages

CU : Gerer utilisateurs

Acteur : ADMIN | Precondition : Role ADMIN

Declencheur : CRUD page admin users

Scenario nominal : CRUD via /api/users avec validation et BCrypt

Scenarios alternatifs : Desactivation au lieu de suppression

Postcondition : Base users mise a jour

CU : Recevoir alertes proactives

Acteur : DRIVER | Precondition : app.driver.alerts.enabled=true

Declencheur : DriverProactiveAlertScheduler (5 min)

Scenario nominal : Analyse missions -> prediction retard -> notification si risque

Scenarios alternatifs : Dedup 30 min -> pas de doublon notification

Postcondition : Alerte visible dashboard chauffeur

CHAPITRE 6 - DIAGRAMME DE CLASSES

Figure 6.1 - Diagramme de classes metier

```
User 1---* Task (client)     User 1---* Task (driver)
User 1---1 DriverProfile    Order *---1 ProductType
Order 1---0..1 Task         Task 1---* TaskHistory
Task 1---* Notification     User 1---* ChatMessage
AutomationLog (runId, step) Incident *---0..1 Task
Review 1---1 Task (unique)  User 1---* Review
```

6.2 Description des classes importantes

Classe User

Attribut	Type	Contrainte
id	Long	PK
firstName, lastName	String	
email	String	unique
password	String	BCrypt
phone	String	
role	Role enum	5 roles
active	boolean	default true
createdAt, updatedAt	LocalDateTime	

Classe DriverProfile

Attribut	Type	Contrainte
id	Long	PK
user	User	OneToOne
status	DriverStatus	AVAILABLE/BUSY/...
zone	String	
currentLoad	int	
totalDeliveries	int	
performanceScore	BigDecimal	

Classe Order

Attribut	Type	Contrainte
id	Long	PK
client, operator	User	ManyToOne
productType	ProductType	ManyToOne
products	String TEXT	
quantity	int	
amount	BigDecimal	
deliveryAddress	String	
priority	Priority	LOW to CRITICAL
status	OrderStatus	workflow
taskId	Long	FK denormalise

Classe Task

Attribut	Type	Contrainte
id	Long	PK
client, driver, createdBy	User	ManyToOne
order	Order	ManyToOne
title, description	String	
status	TaskStatus	workflow
priority	Priority	
deadline	LocalDateTime	
latitude, longitude	BigDecimal	GPS
proofOfDeliveryImage	String	base64
clientConfirmed	boolean	

Classe ProductType

Attribut	Type	Contrainte
id	Long	PK
code	String	unique
name	String	
description	String	
active	boolean	

Classe TaskHistory

Attribut	Type	Contrainte
id	Long	PK
task	Task	ManyToOne
performedBy	User	ManyToOne
action	String	
oldStatus, newStatus	TaskStatus	

Classe Notification

Attribut	Type	Contrainte
id	Long	PK
user	User	ManyToOne
relatedTask	Task	ManyToOne
type	NotificationType	
title, message	String	
readFlag	boolean	

Classe Incident

Attribut	Type	Contrainte
id	Long	PK
reporter	User	ManyToOne
task, order	Task/Order	ManyToOne
type	IncidentType	
status	IncidentStatus	
description	String	

Classe Review

Attribut	Type	Contrainte
id	Long	PK
client	User	ManyToOne
task	Task	OneToOne unique
rating	int	1-5
comment	String	

Classe ChatMessage

Attribut	Type	Contrainte
id	Long	PK
user	User	ManyToOne
role	MessageRole	USER/ASSISTANT
content	String	
sessionId	String	
intent	String	

Classe AutomationLog

Attribut	Type	Contrainte
id	Long	PK
runId	String	UUID execution
chainType	String	EXPRESS
orderId, taskId	Long	
stepNumber	int	1-7
stepName	String	
status	StepStatus	SUCCESS/SKIPPED/FAILED

6.3 Services backend cles

Classe	Package	Responsabilite
ExpressDeliveryAutomationService	service.automation	Chaine 7 etapes
AIService	service	Facade appels Python
SmartTaskAIClient	client	WebClient HTTP FastAPI
CompositeAIRecommendationService	service.ai	Python + fallback regles
DriverRouteOptimizationService	service.driver	Ordre optimal missions
DriverProactiveAlertService	service.driver	Alertes scheduler
ChatService	service.chat	Orchestration chatbot
AutomationEventListener	event	Declenchement auto HIGH/CRITICAL
SecurityConfig	config	JWT filter chain, CORS, RBAC

CHAPITRE 7 - BASE DE DONNEES

SGBD : PostgreSQL | ORM : Hibernate/JPA | ddl-auto : validate | Migrations : Flyway V1-V10

Table	Description	Migration
users	Comptes 5 roles	V1
driver_profiles	Profil chauffeur 1-1 User	V2
tasks	Missions livraison	V2, V3, V7
task_history	Audit actions taches	V2
notifications	Alertes utilisateur	V2
orders	Commandes clients	V3, V10
incidents	Signalements	V3
reviews	Avis post-livraison	V3
chat_messages	Historique chatbot	V8
automation_logs	Journal chaine IA	V9
product_types	Catalogue 6 types	V10

7.3 Relations JPA

- User 1-N Task (client_id, driver_id, created_by)
- User 1-1 DriverProfile
- Order N-1 ProductType, N-1 User (client, operator)
- Task N-1 Order, 1-N TaskHistory, 1-N Notification
- Review 1-1 Task (contrainte unique task_id)

7.4 Persistance JPA

Entities annotees @Entity, relations @ManyToOne/@OneToOne/@OneToMany. Repositories Spring Data JPA avec methodes derivees et @Query si necessaire. Flyway gere le schema ; Hibernate valide uniquement (pas de auto-ddl).

7.5 Workflows de statuts

OrderStatus : PENDING -> VERIFIED -> PREPARING -> READY -> IN_DELIVERY -> DELIVERED -> RECEIVED

TaskStatus : PENDING/PREPARED -> ASSIGNED -> ACCEPTED -> IN_PROGRESS -> IN_DELIVERY -> COMPLETED

CHAPITRE 8 - BACKEND SPRING BOOT

8.1 Technologies

Technologie	Version	Usage
Java	17	Langage backend
Spring Boot	4.1.0	Framework applicatif
Spring Security	6.x	JWT, RBAC
Spring Data JPA	-	ORM Hibernate
Flyway	-	Migrations SQL
PostgreSQL Driver	-	Connexion BDD
WebFlux WebClient	-	Client HTTP Python
jjwt	0.12.6	Generation/validation JWT
SpringDoc OpenAPI	3.0.1	Documentation Swagger
Lombok	-	Reduction boilerplate
Gradle	-	Build et dependances

8.2 Structure du projet

Arborescence backend (packages principaux)

```
src/main/java/tn/esprit/smarttask/
  controller/     (13 controllers REST)
  service/        (metier + automation + chat + ai + driver)
  repository/     (11 interfaces JPA)
  entity/         (11 entites + enums)
  dto/            (requetes/reponses API)
  security/       (JWT filter, UserDetails)
  config/         (Security, CORS, Swagger, properties)
  client/         (SmartTaskAIClient WebClient)
  event/          (OrderCreatedEvent, listener async)
  exception/      (GlobalExceptionHandler)
src/main/resources/
  application.properties
  db/migration/   (V1 a V10)
```

8.3 Controllers REST

Controller	Base path	Endpoints	Role
AuthenticationController	/api/auth	4	Public/Auth
UserController	/api/users	10	ADMIN/Auth
ManagerController	/api/manager	14	MANAGER
ManagerAIController	/api/manager/ai	8	MANAGER
TaskController	/api/tasks	10	MANAGER
OperatorController	/api/operator	12	OPERATOR
DriverController	/api/driver	15	DRIVER
ClientController	/api/client	10	CLIENT
AutomationController	/api/automation	4	MGR/OP
ProductTypeController	/api/product-types	1	Auth
ChatController	/api/chat	3	Auth

8.3.1 Catalogue complet des API REST

Methode	Endpoint	Securite	Description
POST	/api/auth/register	PUBLIC	Inscription utilisateur
POST	/api/auth/login	PUBLIC	Authentification JWT
POST	/api/auth/logout	PUBLIC	Deconnexion
GET	/api/auth/me	AUTH	Profil courant
GET	/api/users/me	AUTH	Profil utilisateur
PUT	/api/users/me	AUTH	Modifier profil
PATCH	/api/users/me/password	AUTH	Changer mot de passe
GET	/api/users	ADMIN	Lister utilisateurs
GET	/api/users/{id}	ADMIN	Detail utilisateur
POST	/api/users	ADMIN	Creer utilisateur
PUT	/api/users/{id}	ADMIN	Modifier utilisateur
DELETE	/api/users/{id}	ADMIN	Supprimer utilisateur
PATCH	/api/users/{id}/status	ADMIN	Activer/desactiver
PATCH	/api/users/{id}/role	ADMIN	Changer role
GET	/api/manager/dashboard	MANAGER	KPIs manager
GET	/api/manager/drivers	MANAGER	Liste chauffeurs
GET	/api/manager/drivers/{id}	MANAGER	Detail chauffeur
GET	/api/manager/operators	MANAGER	Liste operateurs
GET	/api/manager/clients	MANAGER	Liste clients
GET	/api/manager/delays	MANAGER	Taches en retard
GET	/api/manager/notifications	MANAGER	Notifications
PATCH	/api/manager/notifications/{id}/read	MANAGER	Marquer lu
GET	/api/manager/analytics	MANAGER	Statistiques
GET	/api/manager/analytics/export	MANAGER	Export CSV
GET	/api/manager/ai/recommendations	MANAGER	Recommandations IA
GET	/api/manager/ai/recommend-driver/{taskId}	MANAGER	Recommander chauffeur
GET	/api/manager/incidents	MANAGER	Liste incidents
PATCH	/api/manager/incidents/{id}	MANAGER	Traiter incident
GET	/api/manager/ai/health	MANAGER	Sante service IA
GET	/api/manager/ai/model-status	MANAGER	Etat modeles ML
POST	/api/manager/ai/predict-delay	MANAGER	Prediction retard
POST	/api/manager/ai/predict-delivery-time	MANAGER	Prediction ETA
POST	/api/manager/ai/prioritize-task	MANAGER	Prioriser tache
POST	/api/manager/ai/recommend-driver	MANAGER	Recommander chauffeur
POST	/api/manager/ai/detect-anomaly	MANAGER	Detecter anomalie
POST	/api/manager/ai/analyze	MANAGER	Analyse complete IA
GET	/api/tasks	MANAGER	Lister taches
GET	/api/tasks/{id}	MANAGER	Detail tache
GET	/api/tasks/{id}/history	MANAGER	Historique tache
POST	/api/tasks	MANAGER	Creer tache
PUT	/api/tasks/{id}	MANAGER	Modifier tache
DELETE	/api/tasks/{id}	MANAGER	Supprimer tache
PATCH	/api/tasks/{id}/status	MANAGER	Changer statut
PATCH	/api/tasks/{id}/priority	MANAGER	Changer priorite
POST	/api/tasks/{id}/assign/{driverId}	MANAGER	Assigner chauffeur

POST	/api/tasks/{id}/assign-recommended	MANAGER	Assigner recommande
GET	/api/operator/dashboard	OPERATOR	Dashboard operateur
GET	/api/operator/orders	OPERATOR	Lister commandes
GET	/api/operator/orders/pending	OPERATOR	Commandes en attente
GET	/api/operator/orders/{id}	OPERATOR	Detail commande
POST	/api/operator/orders	OPERATOR	Creer commande
PUT	/api/operator/orders/{id}	OPERATOR	Modifier commande
POST	/api/operator/orders/{id}/verify	OPERATOR	Verifier commande
GET	/api/operator/tasks	OPERATOR	Taches operateur
POST	/api/operator/tasks/{orderId}/prepare	OPERATOR	Preparer commande
GET	/api/operator/clients	OPERATOR	Clients
GET	/api/operator/incidents	OPERATOR	Incidents
POST	/api/operator/incidents	OPERATOR	Signaler incident
GET	/api/driver/dashboard	DRIVER	Dashboard chauffeur
GET	/api/driver/tasks	DRIVER	Missions assignees
GET	/api/driver/tasks/{id}	DRIVER	Detail mission
PATCH	/api/driver/tasks/{id}/accept	DRIVER	Accepter mission
PATCH	/api/driver/tasks/{id}/start	DRIVER	Demarrer livraison
PATCH	/api/driver/tasks/{id}/delivery	DRIVER	En livraison
PATCH	/api/driver/tasks/{id}/complete	DRIVER	Terminer livraison
PATCH	/api/driver/tasks/{id}/location	DRIVER	Mettre a jour GPS
POST	/api/driver/tasks/{id}/proof	DRIVER	Preuve livraison
GET	/api/driver/route	DRIVER	Itineraire missions
GET	/api/driver/route/optimized	DRIVER	Itineraire optimise IA
POST	/api/driver/alerts/check	DRIVER	Verifier alertes IA
POST	/api/driver/incidents	DRIVER	Signaler incident
GET	/api/driver/history	DRIVER	Historique livraisons
GET	/api/driver/performance	DRIVER	Performance chauffeur
GET	/api/client/dashboard	CLIENT	Dashboard client
GET	/api/client/orders	CLIENT	Mes commandes
GET	/api/client/orders/{id}	CLIENT	Detail commande
GET	/api/client/deliveries/{id}	CLIENT	Suivi livraison
GET	/api/client/deliveries/{id}/stream	CLIENT	SSE temps reel
POST	/api/client/deliveries/{id}/confirm	CLIENT	Confirmer reception
POST	/api/client/incidents	CLIENT	Signaler incident
POST	/api/client/reviews	CLIENT	Laisser avis
GET	/api/client/reviews	CLIENT	Mes avis
GET	/api/client/notifications	CLIENT	Notifications
GET	/api/automation/status	MGR/OP	Statut automation
POST	/api/automation/express/{orderId}	MGR/OP	Lancer chaine express
GET	/api/automation/logs	MANAGER	Journal automation
GET	/api/automation/logs/run/{runId}	MGR/OP	Logs par execution
GET	/api/product-types	AUTH	Types produit
GET	/api/chat/init	AUTH	Initialiser chatbot
POST	/api/chat/message	AUTH	Envoyer message chat
GET	/api/chat/history	AUTH	Historique chat

8.11 Securite backend

SecurityFilterChain configure les regles URL. JwtAuthenticationFilter extrait le token Bearer, valide signature et expiration, charge UserDetails. @PreAuthorize sur methodes sensibles. Mots de passe hashes BCryptPasswordEncoder.

Listing 8.1 - Chaine automation (7 etapes)

```
@Transactional(noRollbackFor = AIServiceUnavailableException.class)
public ExpressChainResultResponse execute(Long orderId) {
  step1 READ_ORDER -> step2 AUTO_VERIFY -> step3 CREATE_TASK
  step4 AI_PRIORITIZE -> step5 AI_ASSIGN_DRIVER (score >= 70%)
  step6 ESTIMATE_AND_NOTIFY -> step7 LOG_COMPLETE
}
```

CHAPITRE 9 - FRONTEND ANGULAR

9.1 Architecture Angular

Angular 16 avec composants standalone, lazy loading par role, RxJS pour flux asynchrones. Structure : core/ (auth, services, models), features/ (pages par role), shared/ (chat, navbar).

9.2 Routes Angular

Route	Component	Role requis
/login	LoginComponent	Public
/access-denied	AccessDeniedComponent	Public
/admin/dashboard	AdminDashboardComponent	ADMIN
/admin/users	AdminUsersComponent	ADMIN
/manager/dashboard	ManagerDashboardPageComponent	MANAGER
/manager/tasks	ManagerTasksPageComponent	MANAGER
/manager/planning	ManagerPlanningPageComponent	MANAGER
/manager/map	ManagerMapPageComponent	MANAGER
/manager/drivers	ManagerDriversPageComponent	MANAGER
/manager/delays	ManagerDelaysPageComponent	MANAGER
/manager/incidents	ManagerIncidentsPageComponent	MANAGER
/manager/notifications	ManagerNotificationsPageComponent	MANAGER
/manager/analytics	ManagerAnalyticsPageComponent	MANAGER
/manager/ai	ManagerAiPageComponent	MANAGER
/manager/automation	ManagerAutomationPageComponent	MANAGER
/manager/settings	ManagerSettingsPageComponent	MANAGER
/operator/dashboard	OperatorDashboardPageComponent	OPERATOR
/operator/orders	OperatorOrdersPageComponent	OPERATOR
/operator/orders/create	OperatorOrderCreatePageComponent	OPERATOR
/operator/orders/pending	OperatorPendingPageComponent	OPERATOR
/operator/tasks	OperatorTasksPageComponent	OPERATOR
/operator/tasks/tracking	OperatorTrackingPageComponent	OPERATOR
/operator/incidents	OperatorIncidentsPageComponent	OPERATOR
/operator/settings	OperatorSettingsPageComponent	OPERATOR
/driver/dashboard	DriverDashboardPageComponent	DRIVER
/driver/tasks	DriverTasksPageComponent	DRIVER
/driver/tasks/:id	DriverTaskDetailPageComponent	DRIVER
/driver/route	DriverRoutePageComponent	DRIVER
/driver/incidents	DriverIncidentsPageComponent	DRIVER
/driver/history	DriverHistoryPageComponent	DRIVER
/driver/performance	DriverPerformancePageComponent	DRIVER
/driver/settings	DriverSettingsPageComponent	DRIVER
/client/dashboard	ClientDashboardPageComponent	CLIENT
/client/orders	ClientOrdersPageComponent	CLIENT
/client/orders/:id	ClientOrderDetailPageComponent	CLIENT
/client/delivery/:id	ClientDeliveryPageComponent	CLIENT
/client/notifications	ClientNotificationsPageComponent	CLIENT
/client/reviews	ClientReviewsPageComponent	CLIENT

/client/incidents	ClientIncidentsPageComponent	CLIENT
/client/settings	ClientSettingsPageComponent	CLIENT
/profile	ProfileComponent	AUTH

9.4 Services Angular

- AuthService : login/logout, JWT localStorage
- UserService : profil, gestion users (admin)
- TaskService : CRUD taches manager
- ManagerService : dashboard, analytics, incidents
- OperatorService : commandes, preparation
- DriverService : missions, route optimise, alertes
- ClientService : commandes, SSE livraison, avis
- AiService : predictions via backend (pas Python direct)
- ChatService : init, message, historique
- AutomationService : status, logs, lancer chaine
- ProductTypeService : liste types produit

9.6 Guards et Interceptors

- authGuard : verifie presence token JWT
- roleGuard : verifie role dans route.data.roles
- jwt.interceptor : ajoute header Authorization Bearer

9.8 Flux authentification frontend

Flux Angular -> JWT

```
LoginComponent -> AuthService.login() -> POST /api/auth/login
<- JWT token -> localStorage.setItem('token')
Route protegee -> authGuard OK -> jwt.interceptor ajoute Bearer
-> requetes API securisees -> dashboard selon role
```

9.9 Pages principales par role

Role	Pages cles
Manager	Dashboard KPIs, Taches, Carte, IA, Automation, Analytics
Operator	Commandes, Create (types produit), Pending, Tracking
Driver	Missions, Route optimise IA, Performance, Alertes
Client	Commandes, Delivery SSE, Reviews, Notifications
Admin	Users CRUD, Dashboard

CHAPITRE 10 - INTELLIGENCE ARTIFICIELLE ET MACHINE LEARNING

10.1 Problematique IA

L'IA repond a des problemes predictifs et decisionnels : anticiper les retards, estimer les temps de livraison, recommander le meilleur chauffeur, prioriser les taches urgentes, optimiser les itineraires et assister les utilisateurs.

10.2 Donnees

Dataset : smarttask-ai/data/raw/smarttask_delivery_dataset.csv

Origine : SYNTHETIC_DEV | Echantillons : 3000 | Colonnes : features + cibles delayed et actual_delivery_time

Variable	Type	Signification
distance_km	float	Distance estimee km
deadline_minutes	int	Minutes restantes deadline
priority_score	int	Score priorite encode
hour, day_of_week	int	Contexte temporel
driver_current_tasks	int	Charge chauffeur
driver_delay_rate	float	Taux retard historique
traffic_score	float	Indice trafic
delayed	0/1	Cible classification
actual_delivery_time	int	Cible regression (minutes)

10.3 Pretraitement

- 1. Importation CSV via pandas
- 2. Nettoyage valeurs manquantes
- 3. Encodage variables categorielles (priority, traffic)
- 4. Separation features / cibles
- 5. Split train/test (80/20)
- 6. Standardisation pour modeles sensibles

10.4 Exploration

Statistiques descriptives, distribution retards, correlation features. Scripts dans app/ml/.

10.5 Modeles

Modele	Type	Algorithme	Metriques test
delay_model	Classification	Logistic Regression	Acc 0.965, F1 0.982, AUC 0.990
delivery_time_model	Regression	XGBoost	MAE 6.32 min, R2 0.960
priority_model	-	Stub non entraine	Regles fallback
driver_recommend	Scoring	Rule-based v1.0.0	Pas de .pkl
route_optimize	Heuristique	TSP nearest neighbor	Pas de .pkl
anomaly_detect	Statistique	Seuils AnalyticsService	Pas de .pkl

Note : fichiers .pkl non commites - generer via : python -m app.ml.train_all

10.7 Service Python FastAPI

Structure smarttask-ai

```
smarttask-ai/
  app/main.py           FastAPI entrypoint
```

```
app/api/              predict, chat, route, health
app/services/         PredictionService, RouteOptimizationService
app/ml/               train_delay, train_delivery_time, train_all
app/models/           .pkl + model_metadata.json
app/data/knowledge_base.json   Chatbot
```

10.8 Integration Spring Boot + Python

SmartTaskAIClient utilise WebClient (Spring WebFlux) pour appels HTTP synchrones vers http://localhost:8000. Format JSON request/response. Timeout configurable (app.ai.timeout-seconds=10). CompositeAIRecommendationService bascule sur RuleBasedAIRecommendationService si Python indisponible.

Methode	Endpoint Python	Description
GET	/api/ai/health	Sante service + version
GET	/api/ai/model/status	Etat chargement modeles ML
GET	/api/ai/metrics	Compteurs predictions
POST	/api/ai/predict/delay	Classification risque retard
POST	/api/ai/predict/delivery-time	Regression ETA minutes
POST	/api/ai/detect/anomaly	Detection anomalies statistiques
POST	/api/ai/recommend/driver	Scoring chauffeur rule-based
POST	/api/ai/prioritize/task	Score priorite tache
POST	/api/ai/chat	Reponse chatbot Copilot
GET	/api/ai/chat/suggestions	Suggestions prompts
POST	/api/ai/optimize/route	Optimisation itineraire TSP

10.9 Chaine automation express

Etape	Nom	Action
1	READ_ORDER	Lire commande depuis BDD
2	AUTO_VERIFY	PENDING -> VERIFIED
3	CREATE_TASK	Tache PREPARED, order READY
4	AI_PRIORITIZE	POST /api/ai/prioritize/task
5	AI_ASSIGN_DRIVER	POST /api/ai/recommend/driver si score>=70%
6	ESTIMATE_AND_NOTIFY	ETA + notifications client/chauffeur
7	LOG_COMPLETE	Ecriture automation_logs

10.10 IA Chauffeur

- Route : GET /api/driver/route/optimized -> POST /api/ai/optimize/route
- Alertes : DriverProactiveAlertScheduler interval 300000ms (5 min)
- Config : deadline-warning-minutes=45, delay-risk-minutes=20, dedup-minutes=30

10.11 Chatbot SmartTask Copilot

Intents regex + knowledge_base.json. Contexte role depuis ChatContextBuilderService (stats dashboard injectees). Fallback Java ChatFallbackService si Python down.

CHAPITRE 11 - DIAGRAMMES DE SEQUENCE

Figure 11.1 - Authentification JWT

```
User -> Angular: saisir credentials
Angular -> Spring Boot: POST /api/auth/login
Spring Boot -> PostgreSQL: SELECT user, verify BCrypt
Spring Boot -> Angular: JWT token + user info
Angular -> localStorage: store token
Angular -> Spring Boot: requetes + Authorization Bearer
```

Figure 11.2 - Commande HIGH + Automation

```
Operator -> Angular -> Spring: POST /api/operator/orders (priority HIGH)
Spring -> PostgreSQL: INSERT order
Spring -> EventPublisher: OrderCreatedEvent
AutomationEventListener (@Async) -> ExpressDeliveryAutomationService
  -> 7 etapes (verify, task, IA, assign, notify, log)
Spring -> Angular: OrderResponse
```

Figure 11.3 - Prediction retard IA

```
Manager -> Angular -> Spring: POST /manager/ai/predict-delay
Spring -> AIService -> SmartTaskAIClient
WebClient -> Python: POST /api/ai/predict/delay
Python -> delay_model.pkl: predict_proba
Python -> Spring: delayProbability, riskLevel, modelVersion
Spring -> Angular: JSON affiche page Manager IA
```

Figure 11.4 - Route optimisee chauffeur

```
Driver -> Angular -> Spring: GET /api/driver/route/optimized
Spring -> DriverRouteOptimizationService
WebClient -> Python: POST /api/ai/optimize/route (missions GPS)
Python -> TSP heuristic: ordre optimal
Python -> Spring: orderedTaskIds, totalDistanceKm
Spring -> Angular: carte/liste reordonnee
```

Figure 11.5 - Chatbot Copilot

```
User -> ChatWidget -> Angular ChatService
POST /api/chat/message -> ChatService
  -> SmartTaskAIClient POST /api/ai/chat (ou fallback Java)
  -> INSERT chat_messages (USER + ASSISTANT)
<- reponse affichee widget
```

11.6 Gestion des erreurs

GlobalExceptionHandler intercepte exceptions metier et validation. AIServiceUnavailableException : pas de rollback transaction automation. Frontend affiche messages erreur HTTP via interceptors/services.

CHAPITRE 12 - OUTILS ET TECHNOLOGIES

Couche	Technologie	Version	Role
Backend	Java / Spring Boot	17 / 4.1.0	API REST metier
Backend	Spring Security + JWT	jjwt 0.12.6	Auth stateless
Backend	Spring Data JPA / Flyway	-	ORM + migrations
Frontend	Angular / TypeScript	16.2 / 5.1	SPA multi-role
Frontend	RxJS	7.8	Programmation reactive
BDD	PostgreSQL	14+	Persistance relationnelle
IA	Python / FastAPI / Uvicorn	3.11+	Microservice ML
IA	scikit-learn / XGBoost / pandas	1.5+/2.1+	ML pipeline
Build	Gradle / npm	-	Compilation
Doc	SpringDoc OpenAPI	3.0.1	Swagger UI
Tests	JUnit 5 / pytest / Karma	-	Tests unitaires

CHAPITRE 13 - TESTS

Type	Fichier / Outil	Contenu
Unitaire Java	AuthenticationControllerTest	Login register
Unitaire Java	ManagerAIControllerTest	Endpoints IA manager
Unitaire Java	AutomationControllerTest	Chaine automation
Securite Java	ManagerSecurityTest, TaskControllerSecurityTest	RBAC
Integration	SmartTaskApplicationTests	Context Spring
Python	test_api.py, test_chat.py	14 tests endpoints
Angular	app.component.spec.ts	Test bootstrap
Manuel	SmartTask-API.postman_collection.json	Collection Postman

Profil test : application-test.properties avec H2 in-memory, Flyway desactive.

CHAPITRE 14 - SECURITE

- JWT expiration 86400000 ms (24h), secret configurable app.jwt.secret
- BCrypt pour hash mots de passe
- RBAC 5 roles avec SecurityFilterChain + @PreAuthorize
- CORS : app.cors.allowed-origins=http://localhost:4200
- Validation @Valid sur DTO entree
- Frontend : token localStorage, guards, interceptor Bearer
- Endpoints publics limites : /api/auth/register, /api/auth/login, swagger

CHAPITRE 15 - DOCKER ET DEVOPS

Present dans le depot : smarttask-ai/Dockerfile (Python 3.11-slim, port 8000).

NON present : docker-compose.yml, Jenkins, GitHub Actions, Kubernetes, SonarQube, Prometheus.

Fonctionnalite prevue / future amelioration : pipeline CI/CD complet, monitoring Grafana.

CHAPITRE 16 - DEPLOIEMENT

Figure 16.1 - Processus de deploiement possible

```
Development:
  PostgreSQL + gradlew bootRun + uvicorn + ng serve
Build:
  gradlew bootJar + ng build --configuration production
  python -m app.ml.train_all (generer .pkl)
Tests:
  gradlew test + pytest + ng test
Production possible:
  JAR Spring Boot (9090) + fichiers static Angular (nginx)
  + container Python IA + PostgreSQL managed
```

Service	Commande	Port
PostgreSQL	Creer BDD smarttask_db	5432
Backend	gradlew bootRun	9090
Python IA	uvicorn app.main:app --port 8000	8000
Frontend	ng serve	4200

CHAPITRE 17 - SCENARIO GLOBAL JURY

Scenario de demonstration complete :

- 1. Operator se connecte (operator1@smarttask.com)
- 2. Cree commande type MEDICAMENTS, priorite HIGH, adresse livraison
- 3. Automation declenchee : 7 logs SUCCESS dans automation_logs
- 4. Manager consulte page Automation et IA Analytics
- 5. Driver voit mission assignee, itineraire optimise, alertes proactives
- 6. Driver complete livraison avec preuve photo
- 7. Client suit via SSE, confirme reception, laisse avis 5 etoiles
- 8. Chatbot Copilot repond aux questions du manager

CHAPITRE 18 - DIFFICULTES ET SOLUTIONS

Probleme	Cause	Solution	Resultat
Connection refused :8000	Python IA arrete	Demarrer uvicorn	IA operationnelle
Rollback automation	Exception IA en @Transactiona	noRollbackFor AIServiceUnavailable	Logs partiels OK
CORS blocked	Origines differentes	CorsConfiguration 4200	Frontend accede API
.pkl absents	Non commites git	train_all.py	Modeles generes
Doublon automation	Re-trigger manuel	Verification statut order	Refus si deja traite

CHAPITRE 19 - CHOIX TECHNOLOGIQUES

Spring Boot vs alternatives

Retenu pour ecosysteme entreprise, securite mature, JPA, communaute. Alternative Node.js/Django non retenue pour experience Java PFE.

Angular vs React/Vue

Retenu pour structure modulaire, TypeScript, routing/guards natifs, adequation projets entreprise.

PostgreSQL vs MySQL

Retenu pour robustesse, enums, contraintes, adequation JPA/Flyway.

Python pour IA

Standard de facto ML (scikit-learn, XGBoost). Microservice decouple evite surcharge JVM.

JWT vs sessions

Stateless adapte API REST SPA. Pas de session serveur a gerer.

CHAPITRE 20 - LIMITES ET PERSPECTIVES

Limites actuelles

- Dataset ML synthetique (3000 echantillons), pas de donnees reelles
- Modeles .pkl non versionnes dans le depot
- Docker partiel (IA seulement)
- Tests frontend limites (1 spec)
- priority_model non entraine (stub)

Perspec

- Chaines automation 2 et 3 (anti-retard, recovery incident) - prevues
- docker-compose stack complete
- CI/CD GitHub Actions, SonarQube
- Application mobile chauffeur
- Re-entrainement ML sur donnees production
- Deploiement cloud (AWS/Azure)

CHAPITRE 21 - CONCLUSION

Le projet SmartLogix atteint ses objectifs principaux : plateforme logistique multi-acteurs complete, integration IA operationnelle (predictions, automation, chatbot, route chauffeur), architecture moderne 3 tiers avec securite JWT et persistance PostgreSQL.

Competences acquises : conception UML, developpement full-stack, microservices, machine learning, DevOps partiel. L'IA apporte une valeur concrete via l'automation express et l'aide a la decision manager/chauffeur.

Perspectives : industrialisation (CI/CD, Docker complet), enrichissement ML sur donnees reelles, et extension fonctionnelle (mobile, nouvelles chaines automation).

ANNEXE DETAILLEE - SERVICES BACKEND

Service	Methodes / Responsabilite
AuthService	register, login, logout, getCurrentUser
UserService	CRUD users, change password, update profile, role/status
TaskService	CRUD tasks, assign driver, history, status/priority update
ManagerService	dashboard KPIs, drivers, delays, analytics export, inciden
OperatorService	orders CRUD, verify, prepare task, clients list
DriverService	missions lifecycle, location, proof, route, performance
ClientService	orders, delivery tracking SSE, confirm, reviews
IncidentService	create/update incidents multi-acteurs
AIService	facade predict delay, ETA, prioritize, recommend, anomaly
CompositeAIRecommendationService	Python first, fallback RuleBasedAI
RuleBasedAIRecommendationService	regles metier si Python down
ExpressDeliveryAutomationService	chaine 7 etapes transactionnelle
ChatService	init session, send message, history persistence
ChatContextBuilderService	injecte stats role pour contexte IA
ChatFallbackService	reponses Java si Python indisponible
DriverRouteOptimizationService	appel Python optimize/route
DriverProactiveAlertService	analyse missions, notifications alerte
ProductTypeService	liste types actifs depuis product_types
RealTimeEventService	publication evenements SSE livraison
JwtService	generation/validation token JWT
CustomUserDetailsService	charge UserDetails pour Spring Security

Detail AuthService et JwtService

AuthService recoit LoginRequest/RegisterRequest, delegue a UserRepository pour verification BCrypt, genere JWT via JwtService (claims: subject=email, expiration 24h). Logout cote stateless invalide token cote client uniquement.

Listing - JwtAuthenticationFilter (concept)

```
1. Extraire header Authorization: Bearer <token>
2. JwtService.validateToken(token)
3. Charger UserDetails via CustomUserDetailsService
4. SecurityContextHolder.setAuthentication(...)
5. Continuer filter chain
```

Detail TaskService

TaskService gere le cycle complet : creation depuis Order (OperatorService.prepare), assignation manuelle ou via assign-recommended (appel IA), transitions statut avec TaskHistory audit, notifications via NotificationRepository.

Detail OperatorService - creation commande

OrderRequest contient productTypeId et customProductType optionnel (si Autre). Validation : client existant, priorite enum, adresse obligatoire. Apres save, publication OrderCreatedEvent si priorite HIGH/CRITICAL.

ANNEXE DETAILLEE - DTO (Data Transfer Objects)

Les DTO separent le contrat API du modele JPA. Validation Bean Validation (@NotNull, @Email, @Size) sur les Request. Response objects evitent exposition mot de passe.

- LoginRequest, RegisterRequest
- UserRequest, UpdateProfileRequest
- PasswordChangeRequest, RoleUpdateRequest
- OrderRequest (productTypeId, customProductType)
- TaskRequest, TaskStatusUpdateRequest
- PriorityUpdateRequest, StatusUpdateRequest
- IncidentRequest, IncidentUpdateRequest
- ReviewRequest, DeliveryConfirmRequest
- ProofOfDeliveryRequest (image base64)
- LocationUpdateRequest (lat/lng)
- ChatSendRequest
- TaskIdRequest
- AIDelayPredictionRequest, AIDeliveryTimePredictionRequest
- AITaskPrioritizationRequest, AIDriverRecommendationRequest
- AIChatRequest, AIRouteOptimizationRequest

Responses principales

- LoginResponse (token, user)
- UserResponse, PageResponse<T>
- OrderResponse, TaskResponse, TaskHistoryResponse
- ManagerDashboardResponse, OperatorDashboardResponse
- DriverDashboardResponse, ClientDashboardResponse
- AIDelayPredictionResponse, AIDeliveryTimePredictionResponse
- ExpressChainResultResponse, AutomationLogResponse
- ChatReplyResponse, ChatInitResponse, DeliveryTrackingResponse
- DriverRouteOptimizationResponse, ApiErrorResponse

ANNEXE DETAILLEE - COMPOSANTS ANGULAR (49)

Composant	Chemin	Role
LoginComponent	auth/login	Formulaire authentification
AccessDeniedComponent	access-denied	Page acces refuse
ProfileComponent	profile	Profil utilisateur
AdminDashboardComponent	admin	Dashboard admin
AdminUsersComponent	admin/users	CRUD utilisateurs
ManagerLayoutComponent	manager	Layout sidebar manager
ManagerDashboardPageComponent	manager/dashboard	KPIs manager
ManagerTasksPageComponent	manager/tasks	Liste/gestion taches
ManagerPlanningPageComponent	manager/planning	Planning livraisons
ManagerMapPageComponent	manager/map	Carte missions
ManagerDriversPageComponent	manager/drivers	Liste chauffeurs
ManagerDelaysPageComponent	manager/delays	Taches en retard
ManagerIncidentsPageComponent	manager/incidents	Gestion incidents
ManagerNotificationsPageComponent	manager/notifications	Notifications
ManagerAnalyticsPageComponent	manager/analytics	Graphiques analytics
ManagerAiPageComponent	manager/ai	Predictions IA manager
ManagerAutomationPageComponent	manager/automation	Journal chaine IA
ManagerSettingsPageComponent	manager/settings	Parametres
OperatorLayoutComponent	operator	Layout operateur
OperatorDashboardPageComponent	operator/dashboard	Dashboard operateur
OperatorOrdersPageComponent	operator/orders	Liste commandes
OperatorOrderCreatePageComponent	operator/orders/create	Formulaire + types produit
OperatorPendingPageComponent	operator/orders/pending	Commandes en attente
OperatorTasksPageComponent	operator/tasks	Taches preparation
OperatorTrackingPageComponent	operator/tasks/tracking	Suivi preparation
OperatorIncidentsPageComponent	operator/incidents	Incidents operateur
OperatorSettingsPageComponent	operator/settings	Parametres
DriverLayoutComponent	driver	Layout chauffeur
DriverDashboardPageComponent	driver/dashboard	Dashboard + alertes IA
DriverTasksPageComponent	driver/tasks	Liste missions
DriverTaskDetailPageComponent	driver/tasks/:id	Detail mission
DriverRoutePageComponent	driver/route	Itineraire optimise IA
DriverIncidentsPageComponent	driver/incidents	Signaler incident
DriverHistoryPageComponent	driver/history	Historique livraisons
DriverPerformancePageComponent	driver/performance	Score performance
DriverSettingsPageComponent	driver/settings	Parametres
ClientLayoutComponent	client	Layout client
ClientDashboardPageComponent	client/dashboard	Dashboard client
ClientOrdersPageComponent	client/orders	Mes commandes
ClientOrderDetailPageComponent	client/orders/:id	Detail commande
ClientDeliveryPageComponent	client/delivery/:id	Suivi SSE livraison
ClientNotificationsPageComponent	client/notifications	Notifications
ClientReviewsPageComponent	client/reviews	Mes avis
ClientIncidentsPageComponent	client/incidents	Signaler probleme

ClientSettingsPageComponent	client/settings	Parametres
ChatWidgetComponent	shared/chat-widget	Widget chatbot flottant
NavbarComponent	shared/navbar	Barre navigation
DashboardComponent	shared/dashboard	Composant dashboard generique
AppComponent	app	Racine application

ANNEXE DETAILLEE - PIPELINE MACHINE LEARNING

Comparaison algorithmes - Modele retard (classification)

Algorithme	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (retenu)	0.965	1.000	0.965	0.982	0.990
Random Forest	0.990	0.993	0.997	0.995	0.959
XGBoost	0.993	0.993	1.000	0.997	0.967

Logistic Regression retenu pour interpretabilite et performance suffisante (F1=0.982). Matrice confusion test : TN=4, FP=0, FN=21, TP=575.

Comparaison algorithmes - Modele ETA (regression)

Algorithme	MAE (min)	RMSE (min)	R2
Linear Regression	7.45	9.43	0.945
Random Forest	7.14	8.93	0.950
XGBoost (retenu)	6.32	8.02	0.960

Scripts d'entrainement

- generate_dataset.py : genere 3000 echantillons synthetiques
- train_delay.py : compare LR, RF, XGBoost, sauve delay_model.pkl
- train_delivery_time.py : compare LR, RF, XGBoost, sauve delivery_time_model.pkl
- train_priority.py : stub (non entraine)
- train_all.py : pipeline complet + model_metadata.json

Inference en production

Flux inference Python

```
PredictionService.load_models() au demarrage FastAPI
POST /predict/delay:
  1. Valider DelayPredictionRequest (Pydantic)
  2. Feature engineering (priority_score, traffic_score)
  3. model.predict_proba(features)
  4. Retourner delayProbability, riskLevel, modelVersion
```

Evaluation et metriques

Classification : accuracy, precision, recall, F1-score, ROC-AUC, matrice confusion. Regression : MAE, RMSE, R2. Split train/test 80/20 sur dataset synthetique.

ANNEXE - EXEMPLES JSON API

Exemple POST /api/auth/login - Request

```
{
  "email": "manager@smarttask.com",
  "password": "Manager123!"
}
```

Exemple POST /api/auth/login - Response

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": { "id": 2, "email": "manager@smarttask.com", "role": "MANAGER" }
}
```

Exemple POST /api/operator/orders - Request

```
{
  "clientId": 5,
  "productTypeId": 4,
  "customProductType": null,
  "products": "Medicaments urgents",
  "quantity": 1,
  "amount": 150.00,
  "deliveryAddress": "12 Rue Example, Tunis",
  "priority": "HIGH",
  "scheduledDate": "2026-08-30",
  "scheduledTime": "14:00"
}
```

Exemple POST /api/manager/ai/predict-delay - Response

```
{
  "delayProbability": 0.73,
  "riskLevel": "HIGH",
  "modelVersion": "1.0.0",
  "source": "python-ml"
}
```

Exemple POST /api/ai/chat - Request (Python)

```
{
  "message": "Combien de livraisons en retard ?",
  "role": "MANAGER",
  "context": [{ "key": "delayed_tasks", "value": "3" }]
}
```

ANNEXE DETAILLEE - SECURITE

Matrice d'acces par role

Ressource	ADMIN	MANAGER	OPERATOR	DRIVER	CLIENT
/api/users CRUD	OUI	NON	NON	NON	NON
/api/manager/*	NON	OUI	NON	NON	NON
/api/operator/*	NON	NON	OUI	NON	NON
/api/driver/*	NON	NON	NON	OUI	NON
/api/client/*	NON	NON	NON	NON	OUI
/api/automation/*	NON	OUI	OUI*	NON	NON
/api/chat/*	OUI	OUI	OUI	OUI	OUI
/api/product-types	OUI	OUI	OUI	OUI	OUI

* Operator : status et express trigger uniquement, pas logs complets.

Bonnes pratiques implementees

- Separation credentials : password jamais dans Response DTO
- Validation entree : @Valid sur controllers
- Gestion erreurs : ApiErrorResponse structure (code, message, timestamp)
- CORS restreint a localhost:4200 en dev
- open-in-view=false : pas de lazy loading hors transaction

Recommandations production

- Changer app.jwt.secret (256 bits minimum)
- HTTPS obligatoire
- Rate limiting sur /api/auth/login

Fonctionnalite prevue / future amelioration : refresh token, 2FA, audit log securite.

ANNEXE DETAILLEE - MIGRATIONS FLYWAY

Version	Objet	Detail
V1	users	id PK, first_name, last_name, email UNIQUE, password, phon
V2	driver_profiles	user_id FK UNIQUE, status, zone, current_load, performance
V2	tasks	client_id, driver_id, created_by FK, title, status, priori
V2	task_history	task_id FK, performed_by FK, action, old/new status
V2	notifications	user_id FK, related_task_id FK, type, title, message, read
V3	orders	client_id, operator_id FK, products TEXT, amount, status,
V3	incidents	reporter_id, task_id, order_id FK, type, status, descripti
V3	reviews	client_id, task_id FK UNIQUE, rating 1-5, comment
V4	orders	Ajout statut RECEIVED dans CHECK constraint
V5	seed	Commande DELIVERED test pour client1@smarttask.com
V6	indexes	Composite idx tasks(status,priority), orders(status), inci
V7	tasks	Colonne proof_of_delivery_image TEXT
V8	chat_messages	user_id FK, role, content, session_id, intent, created_at
V9	automation_logs	run_id, chain_type, order_id, task_id, step_number, step_n
V10	product_types	6 types seeds + product_type_id FK sur orders

ANNEXE DETAILLEE - PLAN DE TESTS

Test	Entree	Resultat attendu	Type
Login valide	manager@smarttask.com / Manager123!	JWT 200	Manuel/JUnit
Login invalide	email wrong / pwd wrong	401 Unauthorized	JUnit
Acces ADMIN users	Token MANAGER sur /api/users	403 Forbidden	Security test
Creer commande HIGH	OrderRequest priority HIGH	Order + automation logs	Manuel
Predict delay	taskId existant	delayProbability 0-1	ManagerAIControllerTest
Automation manuelle	POST express/{orderId}	7 steps SUCCESS/SKIPPED	AutomationControllerTest
Python health	GET /api/ai/health	status ok	pytest test_api
Chat message	POST /api/chat/message	reply + intent	pytest test_chat
Route optimize	2+ missions driver	orderedTaskIds	Manuel
SSE delivery	GET stream client	events status updates	Manuel
Creer avis	rating 5 + taskId	Review saved unique	Manuel
Product types	GET /api/product-types	6 types actifs	Manuel

Build et compilation

- Backend : gradlew compileJava test (Java 17)
- Frontend : ng build --configuration production
- Python : pytest smarttask-ai/tests/

CHAPITRE COMPLEMENTAIRE - UX/UI

Navigation et ergonomie

Chaque role dispose d'un layout dedie (sidebar) avec routes lazy-loaded. Le ChatWidgetComponent est accessible globalement pour assistance contextuelle.

- Manager : navigation 12 pages, accent sur IA et automation
- Operator : workflow guide creation commande avec dropdown types produit
- Driver : actions mission en un clic (accept, start, complete)
- Client : suivi livraison visuel avec timeline et SSE

Feedback utilisateur

- Messages erreur HTTP affiches dans composants
- Validation formulaires cote Angular + backend @Valid
- Indicateurs chargement sur appels API (RxJS finalize)
- Notifications in-app via NotificationResponse

Design system

Styles globaux dans smarttask-frontend/src/styles. Composants shared reutilisables (navbar, dashboard, chat-widget). Information non identifiee dans les fichiers fournis pour framework CSS externe (Bootstrap/Tailwind) - styles custom observes.

Fonctionnalite prevue / future amelioration : mode sombre, internationalisation i18n, application mobile responsive native.

SCENARIO DETAILLE DE SOUTENANCE (DEMO JURY)

Etape 1

Demarrer PostgreSQL, Spring Boot (9090), Python IA (8000), Angular (4200)

Etape 2

Operator login : operator1@smarttask.com / Operator123!

Etape 3

Creer commande : type MEDICAMENTS, priorite HIGH, client existant

Etape 4

Observer logs backend : OrderCreatedEvent -> automation 7 etapes

Etape 5

Manager login : consulter /manager/automation -> 7 logs SUCCESS

Etape 6

Manager /manager/ai : predire retard sur tache creee

Etape 7

Driver login : voir mission assignee, page Route optimisee

Etape 8

Driver : accepter, demarrer, uploader preuve, completer livraison

Etape 9

Client login : suivi SSE /client/delivery/{id}

Etape 10

Client : confirmer reception, avis 5 etoiles

Etape 11

Tous roles : tester ChatWidget Copilot avec question metier

Etape 12

Swagger : demontrer documentation OpenAPI /api-docs

GLOSSAIRE TECHNIQUE

Terme	Definition
API REST	Interface HTTP JSON entre frontend et backend
JWT	JSON Web Token - authentification stateless
RBAC	Role-Based Access Control - controle acces par role
JPA	Java Persistence API - ORM Hibernate
ORM	Object-Relational Mapping - mapping objet/relationnel
DTO	Data Transfer Object - objet transfert API
SSE	Server-Sent Events - flux temps reel unidirectionnel
ML	Machine Learning - apprentissage automatique
FastAPI	Framework Python API async base sur Pydantic
WebClient	Client HTTP reactif Spring WebFlux
Flyway	Outil migrations SQL versionnees
TSP	Traveling Salesman Problem - optimisation itineraire
SPA	Single Page Application - Angular
BCrypt	Algorithme hash mots de passe
CORS	Cross-Origin Resource Sharing
CRUD	Create Read Update Delete
ETA	Estimated Time of Arrival - temps estime arrivee
ROC-AUC	Metrique classification - aire sous courbe ROC
F1-Score	Moyenne harmonique precision et recall
Microservice	Service autonome deployable independamment

ANNEXE - ARBORESCENCE PROJET

Structure racine ProjetPidev

```
ProjetPidev/
  Backend Smarttask/SmartTask/     Spring Boot (Gradle, Java 17)
  smarttask-frontend/              Angular 16 SPA
  smarttask-ai/                    Python FastAPI microservice IA
  docs/                            Scripts rapport + PDF generes
```

Structure smarttask-ai

```
smarttask-ai/
  app/main.py
  app/api/           health, predict, chat, route
  app/services/      PredictionService, RouteOptimizationService
  app/ml/            train_delay, train_delivery_time, train_all
  app/models/        .pkl + model_metadata.json
  app/data/          knowledge_base.json
  tests/             test_api.py, test_chat.py
  Dockerfile
  requirements.txt
```

Structure smarttask-frontend

```
smarttask-frontend/src/app/
  core/              auth, guards, interceptors, services, models
  features/
    admin/           dashboard, users
    manager/           12 pages + layout
    operator/          8 pages + layout
    driver/            8 pages + layout
    client/            8 pages + layout
    auth/              login
  shared/              chat-widget, navbar, dashboard
  app.routes.ts
```

ANNEXES

Annexe A - Comptes de test

Email	Mot de passe	Role
admin@smarttask.com	Admin123!	ADMIN
manager@smarttask.com	Manager123!	MANAGER
operator1@smarttask.com	Operator123!	OPERATOR
driver1@smarttask.com	Driver123!	DRIVER
client1@smarttask.com	Client123!	CLIENT

Annexe B - Types produit (V10)

Code	Nom
DOCUMENTS	Documents et courrier
COLIS_STANDARD	Colis standard
COLIS_FRAGILE	Colis fragile
MEDICAMENTS	Medicaments
EQUIPEMENT	Equipement lourd
ALIMENTAIRE	Produits alimentaires

Annexe C - Configuration application.properties

- server.port=9090
- spring.datasource.url=jdbc:postgresql://localhost:5432/smarttask_db
- app.ai.service-url=http://localhost:8000
- app.automation.express.enabled=true
- app.automation.express.auto-trigger-priorities=HIGH,CRITICAL
- app.driver.alerts.enabled=true
- app.driver.alerts.interval-ms=300000

Annexe D - URLs utiles

- Frontend : http://localhost:4200
- Swagger : http://localhost:9090/swagger-ui.html
- Python docs : http://localhost:8000/docs

Annexe E - Enums principaux

Role: ADMIN, MANAGER, OPERATOR, DRIVER, CLIENT

Priority: LOW, MEDIUM, HIGH, CRITICAL

TaskStatus: PENDING, PREPARED, ASSIGNED, ACCEPTED, IN_PROGRESS, IN_DELIVERY, COMPLETED, CANCELLED

OrderStatus: PENDING, VERIFIED, PREPARING, READY, IN_DELIVERY, DELIVERED, RECEIVED, CANCELLED

REMERCIEMENTS

Je tiens a exprimer ma profonde gratitude a toutes les personnes qui ont contribue a la realisation de ce projet de fin d'etudes.

Mes remerciements s'adressent en premier lieu a mon encadrant [A completer] pour son accompagnement, ses conseils et sa disponibilite tout au long du developpement de la plateforme SmartLogix.

Je remercie egalement l'equipe pedagogique de [Etablissement - A completer] pour la formation de qualite qui m'a permis d'acquerir les competences necessaires en genie logiciel, architecture distribuee et intelligence artificielle.

Enfin, je remercie ma famille et mes proches pour leur soutien moral et leur encouragement durant cette periode exigeante.